

**CENTRO UNIVERSITÁRIO DE BRASÍLIA
FACULDADE DE TECNOLOGIA E CIÊNCIAS SOCIAIS APLICADAS
CURSO DE CIÊNCIA DA COMPUTAÇÃO
SISTEMAS EM TEMPO REAL E EMBARCADOS
PROFESSOR TIAGO LEITE PEREIRA**

**PROJETO FINAL: SISTEMAS EMBARCADOS COM MQTT
SISTEMA DE MONITORAMENTO & ALERTA DE SUPERAQUECIMENTO**

Desenvolver um sistema embarcado que execute tabelas periódicas, reaja a eventos externos, acione atuadores e se comunique por MQTT usando broker público. O sistema desenvolvido simula uma solução de monitoramento de temperatura para ambientes críticos, como salas de servidores, centros de processamento de dados e laboratórios de informática. Nesses ambientes, o aumento excessivo da temperatura pode provocar falhas em equipamentos, interrupção de serviços e prejuízos operacionais.

ALUNOS

22303614 - Ivan Vieira Delmondez de Oliveira

Sumário

1. VISÃO GERAL.....	3
2. OBJETIVOS.....	3
2.1. Objetivo Geral.....	3
2.2. Objetivo Específico.....	3
3. ARQUITETURA DO SISTEMA.....	4
4. COMPONENTES UTILIZADOS.....	4
5. FUNCIONAMENTO DO SISTEMA.....	5
6. ESCALONAMENTO DAS TAREFAS.....	5
7. COMUNICAÇÃO MQTT.....	5
8. MAPA DE TÓPICOS MQTT e PAYLOADS.....	6
9. ESTRATÉGIA DE RECONEXÃO.....	6
10. TESTES REALIZADOS.....	7
10.1. Conexão Wi-Fi.....	7
10.2. Comunicação MQTT.....	7
10.4. Comandos Remotos.....	7
11. CÓDIGO-FONTE FINAL.....	7
12. EVIDÊNCIA DE EXECUÇÃO E DEMONSTRAÇÃO.....	19
12.1. Circuito Montado no Wokwi.....	20
12.2. Inicialização e conexão Wi-Fi/MQTT.....	21
12.3. Publicação de Telemetria.....	21
12.4. Disparo do Alarme.....	22
13. DESAFIOS E DIFICULDADES.....	22
14. CONCLUSÃO.....	22
15. FERRAMENTAS UTILIZADAS.....	23
16. REFERÊNCIAS BIBLIOGRÁFICAS.....	23

1. VISÃO GERAL

Sistemas de monitoramento remoto são amplamente utilizados em ambientes industriais, comerciais e residenciais para acompanhar variáveis críticas, como temperatura, umidade, pressão e consumo energético. A supervisão contínua dessas variáveis permite detectar condições anormais e agir preventivamente antes que ocorram falhas ou danos.

Neste projeto foi desenvolvido um sistema embarcado baseado no ESP32 capaz de monitorar uma variável de temperatura simulada, gerar alarmes quando limites pré-estabelecidos forem ultrapassados e transmitir informações em tempo real através do protocolo MQTT.

Além do monitoramento, o sistema permite receber comandos remotos para alterar parâmetros operacionais, como o limite de temperatura para disparo do alarme e o acionamento de indicadores visuais.

O projeto foi desenvolvido considerando conceitos fundamentais de Sistemas de Tempo Real, incluindo:

- Execução periódica de tarefas;
- Priorização de atividades críticas;
- Comunicação assíncrona;
- Tolerância a falhas;
- Recuperação automática de conexão.

2. OBJETIVOS

2.1. Objetivo Geral

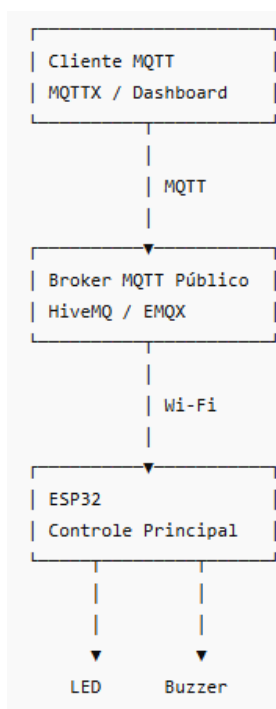
- Desenvolver um sistema embarcado conectado à Internet capaz de monitorar uma variável simulada, publicar telemetria via MQTT e reagir a comandos remotos.

2.2. Objetivo Específico

- Implementar comunicação Wi-Fi utilizando ESP32;
- Utilizar protocolo MQTT para troca de mensagens;
- Simular aquisição periódica de temperatura;
- Gerar alarmes visuais e sonoros;

- Publicar telemetria em formato JSON;
- Receber comandos remotos;
- Implementar reconexão automática ao broker MQTT;
- Aplicar conceitos de sistemas de tempo real utilizando tarefas periódicas baseadas em temporização.

3. ARQUITETURA DO SISTEMA



A arquitetura do sistema é composta por um nó embarcado baseado em ESP32, responsável pela aquisição da temperatura simulada, processamento local das informações, geração de alarmes e comunicação com um broker MQTT público. O dispositivo estabelece conexão com uma rede Wi-Fi e publica periodicamente informações de telemetria em tópicos MQTT específicos. Quando a temperatura ultrapassa o limite configurado, o sistema aciona um LED e um buzzer para sinalização local, além de publicar mensagens de alerta.

4. COMPONENTES UTILIZADOS

Componente	Função
ESP32 DevKit V1	Processamento principal
LED Vermelho	Indicação visual de alarme
Resistor 220	Limitação de corrente do LED
Buzzer	Indicação sonora
Rede Wi-Fi	Comunicação
Broker MQTT	Intermediação das mensagens

5. FUNCIONAMENTO DO SISTEMA

1. O sistema executa continuamente um conjunto de tarefas periódicas.
2. A temperatura simulada varia entre 20°C e 40°C.
3. Quando o valor ultrapassa o limite configurado:
 - a. O LED é acionado;
 - b. O buzzer é ativado;
4. Um alerta MQTT é enviado.
5. Quando o valor retorna à faixa normal:
 - a. O buzzer é desligado;
 - b. O LED retorna ao estado normal.
6. Paralelamente, o sistema publica informações de telemetria periodicamente

6. ESCALONAMENTO DAS TAREFAS

TAREFA	PERÍODO	PRIORIDADE	FUNÇÃO
Controle de Alarme	100 ms	Alta	Avaliação da Condição Crítica
Atualização da Temperatura	500 ms	Média	Simulação do Sensor
Publicação MQTT	2000 ms	Média	Envio de telemetria
Log Serial	1000 ms	Baixa	Monitoramento local
Reconexão MQTT	5000 ms	Alta	Recuperação de falhas

7. COMUNICAÇÃO MQTT

O protocolo MQTT foi escolhido devido ao seu baixo consumo de banda e simplicidade de implementação em sistemas embarcados. O modelo Publish/Subscribe permite desacoplamento entre produtores e consumidores de dados.

Broker Principal	broker.hivemq.com
Porta	1883
Broker Alternativo	broker.emqx.io

8. MAPA DE TÓPICOS MQTT e PAYLOADS

Tópico MQTT	Direção	Finalidade	Exemplo de Payload
ceub/str-embarcados/turma-a/grupo07/esp32-01/telemetria	ESP32 → Broker	Envio periódico dos dados de monitoramento do sistema.	{ "temperatura": 28.5, "limite": 30.0, "alarme": false, "tempo_ms": 15000 }
ceub/str-embarcados/turma-a/grupo07/esp32-01/alerta	ESP32 → Broker	Notificação de situações críticas quando a temperatura ultrapassa o limite configurado.	{ "alerta": "temperatura_alta" }
ceub/str-embarcados/turma-a/grupo07/esp32-01/status	ESP32 → Broker	Indicação do estado operacional do dispositivo e confirmação de conexão ao broker MQTT.	{ "status": "online" }
ceub/str-embarcados/turma-a/grupo07/esp32-01/comando	Broker → ESP32	Recebimento de comandos remotos para controle do sistema.	{ "led": "on" }
ceub/str-embarcados/turma-a/grupo07/esp32-01/comando	Broker → ESP32	Desligamento remoto do LED indicador.	{ "led": "off" }
ceub/str-embarcados/turma-a/grupo07/esp32-01/comando	Broker → ESP32	Alteração remota do limite de temperatura para disparo do alarme.	{ "limite": 25 }

9. ESTRATÉGIA DE RECONEXÃO

Quando ocorre perda de conexão com o broker MQTT:

1. O sistema continua executando todas as tarefas locais;
2. A telemetria deixa de ser enviada temporariamente;
3. O módulo MQTT inicia tentativas automáticas de reconexão;
4. As tentativas são realizadas a cada 5 segundos;
5. Após o restabelecimento da conexão, a comunicação retorna automaticamente.

Essa estratégia evita bloqueios no sistema e garante maior robustez operacional.

10. TESTES REALIZADOS

10.1. Conexão Wi-Fi

- Conexão estabelecida com sucesso.
- Endereço IP obtido automaticamente.

10.2. Comunicação MQTT

- Publicação de mensagens realizada com sucesso.
- Recebimento confirmado pelo cliente MQTT.

10.3. Disparo do Alarme

- LED ativado.
- Buzzer ativado.
- Mensagem de alerta enviada.

10.4. Comandos Remotos

- Alteração do limite de temperatura realizada com sucesso.
- Acionamento remoto do LED validado.

10.5. Falha de Comunicação

- Com a desconexão proposital do broker o sistema continuou executando.
- Reconexão automática realizada.
- Operação retomada sem reinicialização.

11. CÓDIGO-FONTE FINAL

O firmware foi desenvolvido em linguagem C/C++ para a plataforma ESP32 utilizando a Library PubSubClient e contempla os seguintes recursos:

- Conexão Wi-Fi;
- Comunicação MQTT utilizando broker público;
- Publicação de telemetria em formato JSON;
- Recebimento de comandos remotos;
- Acionamento de LED e buzzer;
- Simulação de temperatura;
- Escalonamento de tarefas utilizando `millis()`;

- Reconexão MQTT não bloqueante;
- Tratamento de eventos críticos e geração de alarmes.

(Repositório com código final disponível no [GitHub](#) e nas [Referências](#))

sketch.ino

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>

// =====
// WIFI
// =====

const char* ssid = "Wokwi-GUEST";
const char* password = "";

// =====
// MQTT CLOUD
// =====

const char* mqttServer =
"d796bc3f870f467c85b9fca2b48150e0.s1.eu.hivemq.cloud";

const int mqttPort = 8883;

const char* mqttUser = "Ivan-CEUB";
const char* mqttPassword = "1234Abcd";

// =====
// TOPICOS MQTT
// =====

const char* topicoTelemetria =
"ceub/str-embarcados/turma-a/grupo07/esp32-01/telemetria";

const char* topicoAlerta =
"ceub/str-embarcados/turma-a/grupo07/esp32-01/alerta";
```



```
const char* topicoStatus =
"ceub/str-embarcados/turma-a/grupo07/esp32-01/status";

const char* topicoComando =
"ceub/str-embarcados/turma-a/grupo07/esp32-01/comando";

// =====
// PINOS
// =====

#define LED_PIN 2
#define BUZZER_PIN 15

// =====
// OBJETOS
// =====

WiFiClientSecure espClient;
PubSubClient client(espClient);

// =====
// VARIÁVEIS
// =====

float temperatura = 20.0;
float umidade = 50.0;

float limiteTemperatura = 30.0;

bool alarme = false;
bool ledRemoto = false;

// =====
// TIMERS
// =====

unsigned long tSensor = 0;
```

```
unsigned long tControle = 0;
unsigned long tTelemetria = 0;
unsigned long tLog = 0;
unsigned long tReconexao = 0;

// =====
// CALLBACK MQTT
// =====

void callback(char* topic,
              byte* payload,
              unsigned int length) {

    String msg = "";

    for (unsigned int i = 0; i < length; i++) {
        msg += (char)payload[i];
    }

    Serial.println();
    Serial.println("Comando recebido:");
    Serial.println(msg);

    if (msg.indexOf("\led\":"on\\") >= 0) {
        ledRemoto = true;
    }

    if (msg.indexOf("\led\":"off\\") >= 0) {
        ledRemoto = false;
    }

    int pos = msg.indexOf("limite");

    if (pos >= 0) {

        int inicio = msg.indexOf(":", pos);

        if (inicio > 0) {
```

```
String valor =  
msg.substring(inicio + 1);  
  
valor.replace("}", "");  
valor.replace("\\", "");  
  
limiteTemperatura =  
valor.toFloat();  
  
Serial.print("Novo limite: ");  
Serial.println(limiteTemperatura);  
}  
}  
}  
  
// =====  
// WIFI  
// =====  
  
void conectarWiFi() {  
  
    Serial.println("Conectando WiFi");  
  
    WiFi.begin(ssid, password);  
  
    while (WiFi.status() != WL_CONNECTED) {  
  
        delay(500);  
        Serial.print(".");  
    }  
  
    Serial.println();  
    Serial.println("WiFi conectado");  
  
    Serial.print("IP: ");  
    Serial.println(WiFi.localIP());  
}
```

```
// =====  
// MQTT  
// =====  
  
void conectarMQTT() {  
  
    String clientId =  
        "grupo07-" +  
        String(random(100000));  
  
    Serial.println("Tentando MQTT...");  
  
    if (client.connect(  
        clientId.c_str(),  
        mqttUser,  
        mqttPassword)) {  
  
        Serial.println("MQTT conectado");  
  
        client.subscribe(topicoComando);  
  
        client.publish(  
            topicoStatus,  
            "{\"status\":\"online\"}"  
        );  
  
    } else {  
  
        Serial.print("Falha MQTT.Codigo=");  
        Serial.println(client.state());  
    }  
}  
  
void manterMQTT() {  
  
    if (client.connected()) {
```

```
    client.loop();  
    return;  
}  
  
unsigned long agora = millis();  
  
if (agora - tReconexao >= 5000) {  
  
    tReconexao = agora;  
  
    conectarMQTT();  
}  
}  
  
// =====  
// SETUP  
// =====  
  
void setup() {  
  
    Serial.begin(115200);  
  
    pinMode(LED_PIN, OUTPUT);  
    pinMode(BUZZER_PIN, OUTPUT);  
  
    digitalWrite(LED_PIN, LOW);  
    digitalWrite(BUZZER_PIN, LOW);  
  
    conectarWiFi();  
  
    espClient.setInsecure();  
  
    client.setServer(  
        mqttServer,  
        mqttPort  
    );  
  
    client.setCallback(callback);  
}
```

```
randomSeed(micros());

Serial.println("Sistema iniciado");
}

// =====
// LOOP
// =====

void loop() {

    unsigned long agora = millis();

    // -----
    // MQTT
    // -----

    manterMQTT();

    // -----
    // SENSOR SIMULADO
    // 500 ms
    // -----

    if (agora - tSensor >= 500) {

        tSensor = agora;

        temperatura += 0.5;

        if (temperatura > 40.0) {
            temperatura = 20.0;
        }

        umidade =
            40 + random(0, 500) / 10.0;
    }
```

```
// -----  
// CONTROLE  
// 100 ms  
// -----  
  
if (agora - tControle >= 100) {  
  
    tControle = agora;  
  
    alarme =  
    temperatura >= limiteTemperatura;  
  
    if (alarme) {  
  
        digitalWrite(  
            LED_PIN,  
            HIGH  
        );  
  
        digitalWrite(  
            BUZZER_PIN,  
            HIGH  
        );  
  
    } else {  
  
        digitalWrite(  
            LED_PIN,  
            ledRemoto ? HIGH : LOW  
        );  
  
        digitalWrite(  
            BUZZER_PIN,  
            LOW  
        );  
  
    }  
}
```

```
// -----  
// TELEMETRIA  
// 2 s  
// -----  
  
if (  
    client.connected() &&  
    agora - tTelemetria >= 2000  
) {  
  
    tTelemetria = agora;  
  
    String payload =  
        "{"  
        + "\"temperatura\":"  
        + String(temperatura,1)  
        + ", "  
        + "\"umidade\":"  
        + String(umidade,1)  
        + ", "  
        + "\"limite\":"  
        + String(limiteTemperatura,1)  
        + ", "  
        + "\"alarme\":"  
        + String(alarme ? "true" : "false")  
        + ", "  
        + "\"timestamp_ms\":"  
        + String(agora)  
        + "}";  
  
    bool enviado =  
        client.publish(  
            topicoTelemetria,  
            payload.c_str()  
        );  
  
    if (enviado) {
```



```
    Serial.println();
    Serial.println("Telemetria enviada:");
    Serial.println(payload);

} else {

    Serial.println(
        "Falha ao publicar telemetria"
    );
}

if (alarme) {

    client.publish(
        topicoAlerta,
        "{\"alerta\":\"temperatura_alta\"}"
    );
}
}

// -----
// LOG LOCAL
// 1 s
// -----

if (agora - tLog >= 1000) {

    tLog = agora;

    Serial.println();
    Serial.println("=====");

    Serial.print("Temperatura: ");
    Serial.println(temperatura);

    Serial.print("Umidade: ");
    Serial.println(umidade);
```

```
Serial.print("Limite: ");
Serial.println(limiteTemperatura);

Serial.print("Alarme: ");
Serial.println(
  alarme ? "SIM" : "NAO"
);

Serial.print("MQTT: ");
Serial.println(
  client.connected()
  ? "Conectado"
  : "Desconectado"
);

Serial.println("=====");
}
}
```

diagram.json

```
{
  "version": 1,
  "author": "Ivan Vieira Delmondez de Oliveira",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": -172.8,
      "left": -33.56, "attrs": {} },
    { "type": "wokwi-led", "id": "led1", "top": -90, "left": 186.2,
      "attrs": { "color": "red" } },
    {
      "type": "wokwi-resistor",
      "id": "r2",
      "top": -53.65,
      "left": 96,
      "attrs": { "value": "200" }
    }
  ]
}
```

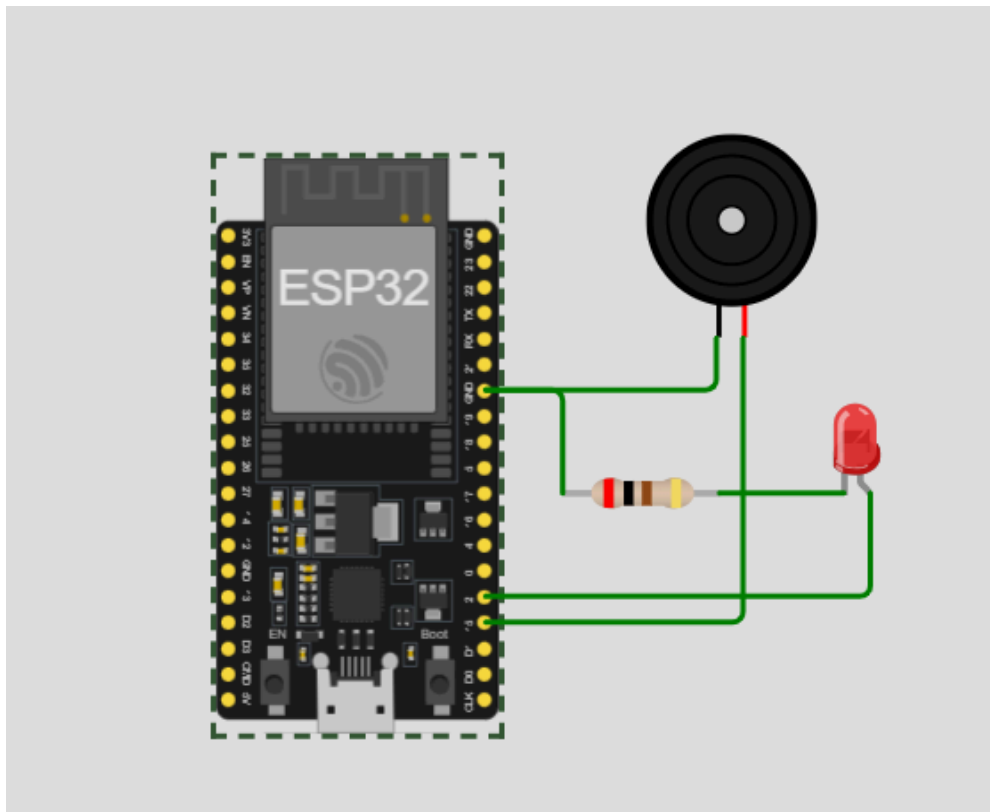
```
    },  
    {  
      "type": "wokwi-buzzer",  
      "id": "bz1",  
      "top": -189.6,  
      "left": 126.6,  
      "attrs": { "volume": "0.1" }  
    }  
  ],  
  "connections": [  
    [ "esp:TX", "$serialMonitor:RX", "", [] ],  
    [ "esp:RX", "$serialMonitor:TX", "", [] ],  
    [ "led1:A", "esp:2", "green", [ "v0" ] ],  
    [ "led1:C", "r2:2", "green", [ "v0" ] ],  
    [ "r2:1", "esp:GND.3", "green", [ "v0" ] ],  
    [ "bz1:2", "esp:15", "green", [ "v0" ] ],  
    [ "bz1:1", "esp:GND.3", "green", [ "v0" ] ]  
  ],  
  "dependencies": {}  
}
```

12. EVIDÊNCIA DE EXECUÇÃO E DEMONSTRAÇÃO

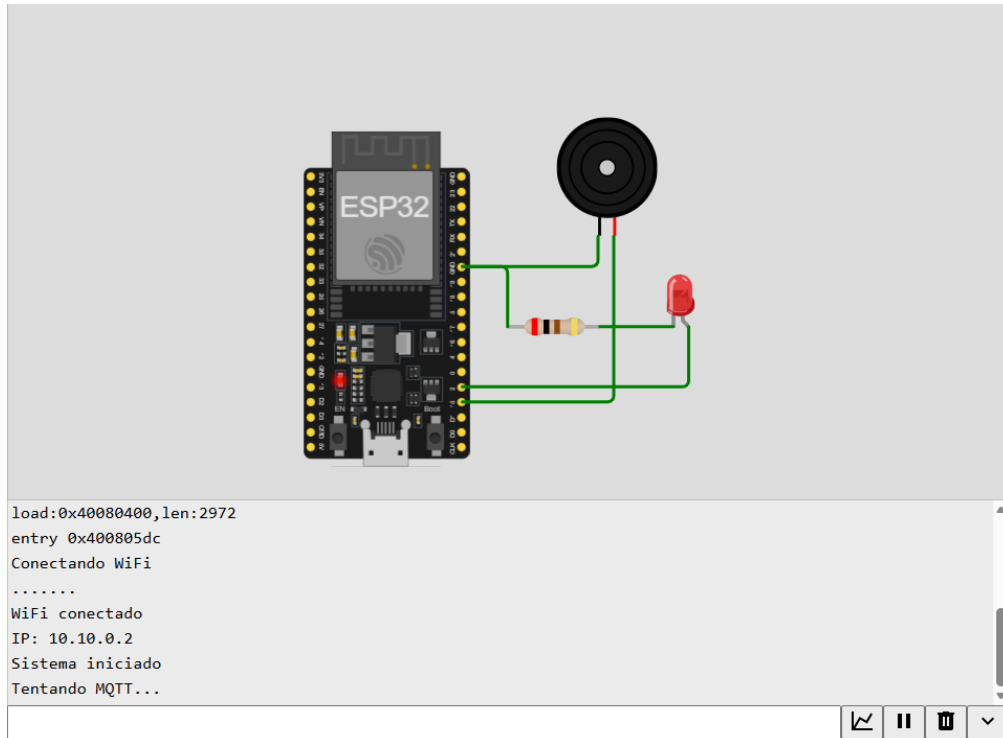
- Inicialização do ESP32;
- Conexão à rede Wi-Fi;
- Conexão ao broker MQTT;
- Execução das tarefas periódicas;
- Simulação da temperatura;
- Acionamento do LED;
- Acionamento do buzzer;
- Publicação de telemetria;
- Estratégia de reconexão em caso de falha.

(Ao clicar em qualquer uma das capturas de tela, será encaminhado a um vídeo de teste. Necessário estar usando uma conta corporativa vinculada ao UniCEUB)

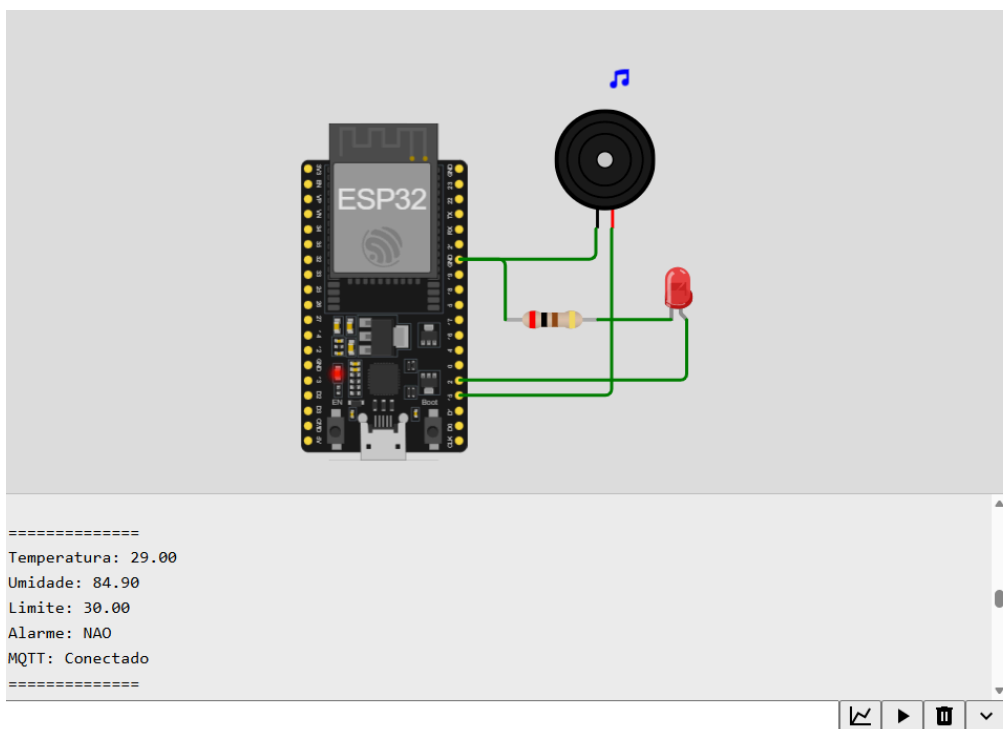
12.1. Circuito Montado no Wokwi



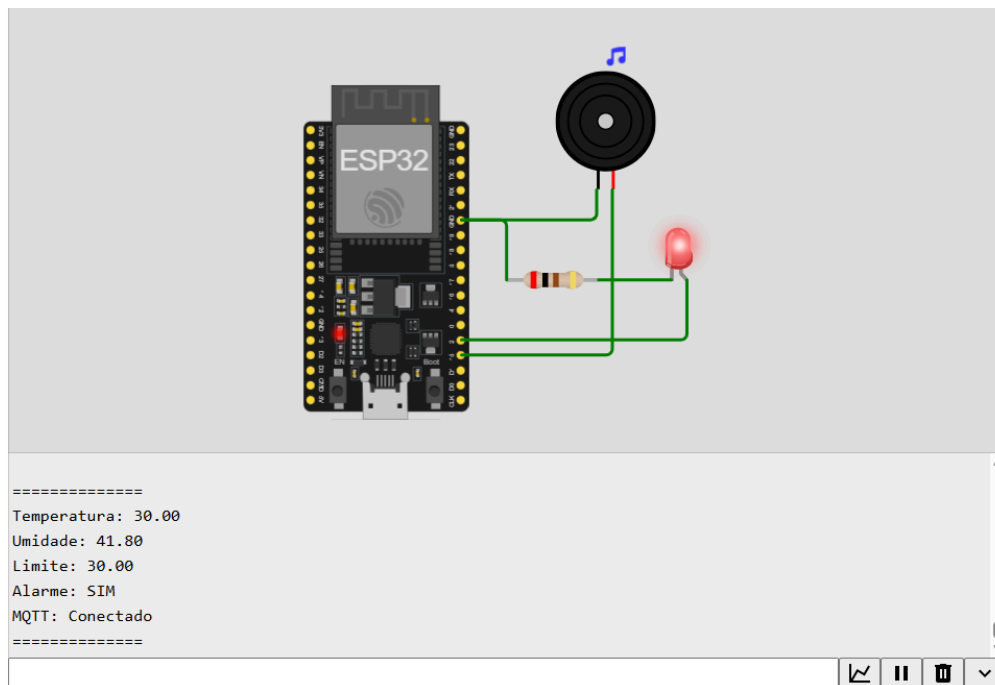
12.2. Inicialização e conexão Wi-Fi/MQTT



12.3. Publicação de Telemetria



12.4. Disparo do Alarme



13. DESAFIOS E DIFICULDADES

Durante o desenvolvimento foram encontradas algumas dificuldades:

- Configuração inicial do ambiente Wokwi;
- Compatibilidade de bibliotecas MQTT;
- Diagnóstico de falhas de conexão;
- Configuração do cliente MQTT para monitoramento dos tópicos;
- Ajuste da lógica de reconexão automática.

Esses desafios contribuíram para um melhor entendimento do funcionamento de sistemas embarcados conectados à Internet.

14. CONCLUSÃO

O projeto atingiu os objetivos propostos ao implementar um sistema embarcado capaz de monitorar uma variável simulada, gerar alarmes locais e realizar comunicação remota por meio do protocolo MQTT. A utilização do ESP32 permitiu integrar recursos de conectividade Wi-Fi e processamento em uma única plataforma, enquanto a arquitetura baseada em tarefas periódicas possibilitou aplicar conceitos fundamentais de Sistemas de Tempo Real.

Os testes realizados demonstraram o correto funcionamento da lógica do sistema, da geração de alarmes, da publicação de telemetria e dos mecanismos de reconexão implementados.

15. FERRAMENTAS UTILIZADAS

Durante o desenvolvimento deste projeto, foram utilizadas as seguintes ferramentas:

- **GitHub**: utilizado para armazenamento e versionamento do código-fonte do projeto.
- **ChatGPT**: utilizado como apoio no desenvolvimento, auxiliando na compreensão de conceitos, construção de expressões regulares e resolução de problemas técnicos.
- **Wokwi**: plataforma de simulação utilizada para desenvolvimento e validação do sistema embarcado baseado em ESP32, permitindo testes sem a necessidade de hardware físico
- **Arduino IDE**: utilizado para implementação do firmware embarcado em linguagem C/C++
- **HiveMQ Public Broker**: broker MQTT utilizado para testes e validação da comunicação entre o ESP32 e os clientes MQTT

16. REFERÊNCIAS BIBLIOGRÁFICAS

Aulas ministradas pelo professor Tiago Leite

Repositório GitHub: [idelmondez/ESP32](https://github.com/idelmondez/ESP32)

OASIS OPEN. MQTT Version 3.1.1. OASIS Standard, 2014. Disponível em: [OASIS MQTT Specification](https://www.oasis-open.org/committees/download.php?ctid=68322). Acesso em: 3 jun. 2026.

ESPRESSIF SYSTEMS. ESP32 Series Datasheet. Disponível em: [ESP32 Documentation](https://www.espressif.com/en/products/modules/esp32). Acesso em: 3 jun. 2026.

KNOLLEARY, Nick et al. PubSubClient Library for Arduino. Disponível em: [PubSubClient GitHub Repository](https://github.com/nkolle/Arduino-PubSubClient). Acesso em: 3 jun. 2026.

WOKWI. Wokwi Simulator Documentation. Disponível em: [Wokwi Documentation](https://wokwi.com/docs/). Acesso em: 3 jun. 2026.